Detailed Summary of the Revised ETH Reports

Demonstrated Speedups, Memory Costs, Failure Cases,
and Requirements for Improving the Full Algorithm

Dr. Muralidhara Subbarao
Technical Report Prepared with the Assistance of AI Models
https://www.integralresearch.net

October 26, 2026

1 Main conclusion

The revised reports [1, 2] advance the aggregated Boolean-jet method from a proved algebraic identity to a tested implementation with measurable speed gains. **They do not yet demonstrate an improvement to the OpenAI paper's complete algorithm or its general complexity bound.**

The strongest demonstrated result is narrower: on matched, small, certified inputs, aggregation preserves every computed coefficient and usually reduces runtime, at the cost of additional coefficient storage.

2 What changed

The paper's Section 3 routine repeatedly evaluates restricted counting problems indexed by pairs of subsets $U \subseteq E$. Its number of restricted calls per interpolation node is

$$C_q(n) = \sum_{j=0}^{\min(q,n)} 2^j \binom{n}{j}. \tag{1}$$

The aggregated method replaces these with

$$B_q(n) = \sum_{j=0}^{\min(q,n)} \binom{n}{j}. \tag{2}$$

It accomplishes this by retaining an additional generating variable that records Hamming-weight information. Contributions from many different E are combined inside one calculation for each U. This is an **exact rearrangement**, not an approximation or a loss of information.

The reduction in calls is proved generally under the stated hypotheses. A runtime reduction does not follow automatically, because each aggregated call performs more coefficient arithmetic.

3 Demonstrated speedups

The implementation compared both routines on **21 instances**, with seven fresh-process timing repetitions per method. Both used the same inputs, exact arithmetic, local assignment scans, and full Hermite reconstruction. Timings included preparation, jet computation, and reconstruction, but excluded process startup and correctness-oracle calculations.

All **294 timed runs, 42 memory runs, and 42 diagnostic runs** agreed on the exact count, every jet coefficient, and the entire reconstructed polynomial. The deformation oracle uses independent enumeration and convolution but shares row construction and a dot-product helper; separate mathematical checks address that shared component.

Representative median results were:

Instance	Variables	Jet order q	Original	Aggregated	Speedup
S04	4	1	4.843 ms	3.756 ms	1.29×
S13	10	3	9.834 s	1.644 s	5.98×
S14	8	4	3.896 s	0.291 s	13.38×
N20	8	3	3.489 s	0.502 s	6.96×
N21	6	4	2.667 s	0.474 s	5.63×

Table 1: Selected matched median timings. Speedup is the original median divided by the aggregated median.

All 18 positive-order cases had lower aggregated medians, ranging from 1.29× **to** 13.38×. Diagnostic counts confirm that less repeated work was performed: for S14, assignment visits fell from **756,862 to 72,698**.

However, these are descriptive measurements on one laptop, with substantial timing variability and no CPU-load or power-management controls. The largest instance had only ten variables. The baseline is our implementation of the paper's formulas, not an OpenAI-supplied executable.

4 Memory costs

Aggregation trades repeated computation for a larger coefficient state. A basic jet has J coefficient slots; the augmented representation can require up to $J(q+1)$ slots per ring element.

Across the tests, whole-pipeline peak Python-traced memory ranged from about **0.9% lower to 3.3% higher** than the original implementation. That small difference partly reflects common preparation and Hermite-recovery allocations dominating the peak.

The kernel-specific difference was larger. For positive-order cases, incremental kernel peaks ranged from approximately 0.98× **to** 1.86× the original:

- S14: 5.54 → 8.65 **KiB**, alongside the largest measured speedup.
- N21: 6.98 → 12.19 **KiB**, alongside a 5.63× speedup.

These are traced-allocation measurements, not total process memory. Interpreter-inclusive working-set peaks were approximately 24.56–25.24 MiB. **There is no demonstrated uniform memory advantage.**

5 Failure cases

Several controls substantially limit how the results should be interpreted:

1. **Zero-order overhead:** when $q = 0$, there is no subset-pair reduction. Aggregation was approximately 16% slower on the empty-formula control, 6% slower on the contradictory-formula control, and nearly tied on the third zero-order case.

2. **A simpler algorithm wins on the first suite:** its 16 instances already separate at the target parameter $T = 0$. Direct shared-summary counting avoids jets entirely and was faster than aggregation in every applicable case. Those later measurements were unpaired, so precise speedup ratios are not justified.

3. **The second suite removes that shortcut, not computational easiness:** five cases change factor assignments between interpolation nodes. They genuinely exercise node-dependent jet recovery, but have only two or three clauses, leaving other simple conventional counting alternatives.

4. **Incorrect shortcuts can destroy exactness:** applying the final alternating functional before truncated products, treating low rank as a small state space, or using uncertified floating-point cancellation can produce wrong answers. These are failures of proposed shortcuts, not errors demonstrated in the OpenAI paper.

5. **The software remains a research prototype:** certain malformed hand-edited node maps are not rejected early, and imported calls require assertions to remain enabled. Neither limitation affected the recorded generated instances.

6 Other proved refinements

The reports retain three additional results that were **not combined with aggregation in these timing comparisons**: weighted-support aggregation under certified higher-order vanishing; removal of redundant summary rows while preserving counting fibers; and target-specific two-adic precision certificates.

The precision tests reduced sufficient extra-bit allowances from **64 to 7**, **144 to 33**, and **96 to 7** in two small interpolation configurations. These are genuine local certificate improvements, not measured savings in the complete recursive algorithm.

7 What remains

Improving the full OpenAI algorithm requires substantially more work:

1. **Implement its actual general construction.** Replace the small input-specific designs with the paper's Section 4 construction, including its construction time, coefficient growth, and measurement-state costs.

2. **Establish robust practical gains.** Test larger and more varied inputs, control timing conditions, and compare against stronger original-loop implementations that share work across calls and use optimized exact arithmetic.

3. **Address Appendix D separately.** The paper's sharper bound uses a different recursive contraction method. Faster Section 3 jets do not automatically improve that recursion.

4. **Integrate precision savings safely.** Preserve the recursion's fixed-lift and divisibility invariants, account for computing recovery weights, and demonstrate savings on nontrivial recursive instances.

5. **Prove a complete improved complexity bound.** Any claimed asymptotic gain must survive all construction, enumeration, arithmetic, recovery, and storage costs. Fewer restricted calls alone is insufficient.

The defensible Ram Calculus contribution is an exact state-aggregation redesign, supported by proofs and finite performance evidence. It is also expressible through conventional generating functions and finite algebra; **no exclusive Ram capability, quaternionic speedup, stronger general SAT exponent, or optimality lower bound has been established.**

References

[1] M. Subbarao. *Ram Calculus and the Exponential Time Hypothesis: Exact Finite-Algebra Reformulations, Aggregated Boolean Jets, and Precision-Certified Counting. Revision 2: Matched Runtime and Memory Benchmarks.* October 26, 2026. Local technical report, `Ram-Calculus-Exponential-Time-Hypothesis-v2.tex` and corresponding PDF. The benchmark investigation is in Section 16.

[2] M. Subbarao. *Summary of the Ram Calculus Exponential-Time-Hypothesis Report: Contributions Relative to the OpenAI Paper, Proved Refinements, Matched Benchmarks, and Open Research Directions.* Revision 2, October 26, 2026. Local technical report, `Summary-RamCal-ETH-report-v2.tex` and corresponding PDF.